

SafeCommit

Technical Documentation

Version 1.0.x

Author: Sushant Kumar Kushwaha
Language: Python
License: MIT License
Repository: GitHub
Package: safecommit-cli

August 2, 2026

Copyright

Copyright © 2026 Sushant Kumar Kushwaha.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, subject to the terms of the MIT License.

Abstract

SafeCommit is an open-source command-line utility designed to identify exposed credentials, secrets, and sensitive information before code reaches a version control system. The project aims to reduce accidental credential leakage by combining curated detection rules with a lightweight scanning engine capable of analysing repositories of varying sizes.

This document serves as the official technical reference for SafeCommit. It describes the project's architecture, implementation, detection engine, supported secret types, command-line interface, testing methodology, design decisions, and future roadmap.

The intended audience includes end users, contributors, maintainers, security researchers, educators, and software engineers interested in understanding the internal operation of SafeCommit.

Preface

Modern software projects interact with numerous external services including cloud providers, source code hosting platforms, package registries, payment gateways, artificial intelligence APIs, continuous integration systems, and third-party infrastructure.

Unfortunately, these integrations frequently require authentication through API keys, access tokens, private keys, certificates, passwords, and other credentials. Once committed into a Git repository, such secrets can rapidly propagate across mirrors, forks, build systems, deployment pipelines, and developer machines, making complete removal exceptionally difficult.

SafeCommit was created to provide developers with a lightweight, fast, and dependency-minimal utility capable of detecting common forms of exposed credentials before they become permanent parts of a project's history.

Rather than attempting to replace enterprise-grade secret management systems, SafeCommit focuses on one objective:

Prevent accidental secret leakage during software development.

Throughout this manual, every major subsystem of SafeCommit is explained in detail, from directory traversal and pattern matching to reporting and testing, with the intention of making the project approachable for both users and future contributors.

Contents

List of Figures	xi
List of Tables	xii
Listings	xiii
1 Introduction	1
1.1 Overview	1
1.2 Motivation	1
1.3 Project Objectives	2
1.3.1 Simplicity	2
1.3.2 Portability	2
1.3.3 Performance	2
1.3.4 Maintainability	3
1.3.5 Accuracy	3
1.4 Design Principles	3
1.4.1 Single Responsibility	3
1.4.2 Explicit Behaviour	3
1.4.3 Minimal Dependencies	4
1.5 Feature Overview	4
1.6 Non-Goals	5
1.7 System Requirements	5
1.8 Typical Workflow	5
1.9 Target Audience	6
2 Installation and Getting Started	7
2.1 Introduction	7
2.2 Supported Installation Methods	7
2.3 Installing from PyPI	7
2.4 Installing from Source	7
2.5 Editable Installation	8
2.6 Creating a Virtual Environment	8
2.6.1 Windows PowerShell	8
2.6.2 Windows Command Prompt	8
2.6.3 Linux and macOS	9
2.7 Updating pip	9
2.8 Project Dependencies	9
2.9 Repository Layout	9

2.10	Verifying Installation	10
2.11	First Scan	10
2.12	Example Session	11
2.13	Exit Codes	11
2.14	Recommended Development Workflow	11
3	Command-Line Interface	13
3.1	Overview	13
3.2	CLI Philosophy	13
3.3	Command Hierarchy	13
3.4	Root Command	14
3.5	Version Command	14
3.6	Scan Command	14
3.7	Command-Line Arguments	14
3.8	Command-Line Options	15
3.9	Execution Lifecycle	15
3.10	Directory Validation	16
3.11	Rich Terminal Output	16
3.12	Severity Classification	16
3.13	Exit Behaviour	17
3.14	Error Handling Philosophy	17
4	Software Architecture	18
4.1	Architectural Overview	18
5	Scanning Engine	19
5.1	Introduction	19
5.2	Design Goals	19
5.3	Responsibilities	19
5.4	Scanner Workflow	20
5.5	Recursive Directory Traversal	20
5.6	File Discovery	21
5.7	Reading Files	21
5.8	Line-Oriented Processing	22
5.9	Pattern Evaluation	22
5.10	Finding Objects	22
5.11	Duplicate Detection	23
5.12	Memory Behaviour	23
5.13	Time Complexity	23
5.14	Error Recovery	24
5.15	Summary	24
6	Detection Engine	25
6.1	Introduction	25

6.2	Detection Philosophy	25
6.3	Detection Pipeline	25
6.4	Detection Rule Architecture	26
6.5	Pattern Categories	26
6.6	Severity Assignment	27
6.6.1	CRITICAL	27
6.6.2	HIGH	27
6.6.3	MEDIUM	27
6.6.4	LOW	28
6.7	Regex Design Principles	28
6.7.1	Specificity	28
6.7.2	Maintainability	28
6.7.3	Performance	28
6.7.4	Compatibility	28
6.8	False Positives	28
6.9	False Negatives	29
6.10	Testing Detection Rules	29
6.11	Extending the Detection Engine	30
7	Supported Credential Types	31
7.1	Introduction	31
7.2	GitHub Personal Access Tokens	31
7.2.1	Overview	31
7.2.2	Assigned Severity	31
7.2.3	Detection Rationale	31
7.2.4	Potential Impact	32
7.2.5	Recommended Remediation	32
7.3	AWS Secret Access Keys	32
7.3.1	Overview	32
7.3.2	Assigned Severity	32
7.3.3	Detection Rationale	33
7.3.4	Potential Impact	33
7.3.5	Recommended Remediation	33
7.4	GitLab Personal Access Tokens	33
7.4.1	Overview	33
7.4.2	Assigned Severity	34
7.4.3	Potential Impact	34
7.4.4	Recommended Response	34
7.5	Stripe Secret Keys	34
7.5.1	Overview	34
7.5.2	Assigned Severity	34
7.5.3	Detection Philosophy	35
7.5.4	Potential Impact	35

7.6	PyPI API Tokens	35
7.6.1	Overview	35
7.6.2	Assigned Severity	35
7.6.3	Security Implications	35
7.6.4	Recommended Remediation	35
7.7	npm Access Tokens	36
7.7.1	Overview	36
7.7.2	Assigned Severity	36
7.7.3	Potential Impact	36
7.8	Hugging Face Access Tokens	36
7.8.1	Overview	36
7.8.2	Assigned Severity	36
7.8.3	Potential Impact	36
7.9	Anthropic API Keys	37
7.9.1	Overview	37
7.9.2	Assigned Severity	37
7.9.3	Potential Impact	37
7.10	Groq API Keys	37
7.10.1	Overview	37
7.10.2	Assigned Severity	37
7.10.3	Security Considerations	37
7.11	Hardcoded Application Secrets	37
7.11.1	Overview	37
7.11.2	Assigned Severity	38
7.11.3	Detection Strategy	38
7.11.4	Security Implications	38
7.12	Generic API Keys and Authentication Tokens	38
7.13	Cryptographic Private Keys	39
7.13.1	Overview	39
7.13.2	Assigned Severity	39
7.13.3	Security Implications	39
7.13.4	Recommended Response	40
7.14	Summary	40
8	Detection Rule Implementation	41
8.1	Introduction	41
8.2	Module Responsibilities	41
8.3	Rule Representation	41
8.4	Regular Expression Compilation	42
8.5	Rule Independence	42
8.6	Matching Strategy	42
8.7	Contextual Detection	42
8.8	Duplicate Suppression	43

8.9	Pattern Naming Conventions	43
8.10	Severity Consistency	44
8.11	Extending the Rule Catalogue	44
8.12	Maintenance Considerations	44
8.13	Summary	45
9	Reporting and Output	46
9.1	Introduction	46
9.2	Reporting Objectives	46
9.3	Finding Presentation	46
9.4	Colour-Based Severity Indicators	47
9.5	Terminal Rendering	47
9.6	Panel Layout	47
9.7	Reported Fields	48
9.8	Successful Scan Output	48
9.9	Detection Summary	48
9.10	Relative versus Absolute Paths	49
9.11	Reporting Multiple Findings	49
9.12	Output Consistency	49
9.13	Machine Readability	50
9.14	HTML Report Generation	50
9.15	Future Reporting Enhancements	51
9.16	Summary	51
10	Testing and Validation	52
10.1	Introduction	52
10.2	Testing Objectives	52
10.3	Testing Strategy	52
10.4	Unit Testing	53
10.5	Integration Testing	53
10.6	Regression Testing	53
10.7	Benchmark Repositories	54
11	Performance Analysis and Benchmarking	55
11.1	Introduction	55
11.2	Performance Objectives	55
11.3	Computational Complexity	55
11.4	Memory Consumption	56
11.5	Scalability	57
11.6	Factors Influencing Performance	57
11.7	Benchmark Methodology	57
11.8	Benchmark Dataset	58
11.9	Example Benchmark Results	58

11.10	Optimisation Opportunities	59
11.11	Performance versus Accuracy	59
11.12	Summary	59
12	Contributing to SafeCommit	60
12.1	Introduction	60
12.2	Ways to Contribute	60
12.3	Development Workflow	60
12.4	Coding Standards	61
12.5	Commit Message Guidelines	61
12.6	Reporting Bugs	62
12.7	Feature Requests	62
12.8	Summary	62
13	Developer Reference	63
13.1	Introduction	63
13.2	<code>__init__.py</code>	63
13.2.1	Purpose	63
13.2.2	Responsibilities	64
13.3	<code>cli.py</code>	64
13.3.1	Purpose	64
13.3.2	Primary Responsibilities	64
13.3.3	Execution Flow	64
13.4	<code>scanner.py</code>	65
13.4.1	Purpose	65
13.4.2	Responsibilities	65
13.4.3	Design Considerations	66
13.5	<code>patterns.py</code>	66
13.5.1	Purpose	66
13.5.2	Responsibilities	66
13.6	<code>utils.py</code>	67
13.6.1	Purpose	67
13.7	Module Dependencies	67
13.8	Design Patterns	67
13.8.1	Separation of Concerns	67
13.8.2	Loose Coupling	67
13.8.3	High Cohesion	67
13.8.4	Extensibility	68
13.9	Code Quality Considerations	68
14	Future Roadmap	69
14.1	Vision	69
14.2	Planned Features	69

14.3 Long-Term Goals	70
Glossary	71
References	72
Bibliography	73
License	74

List of Figures

1.1	High-level SafeCommit scanning workflow.	6
2.1	Repository structure.	10
3.1	SafeCommit command hierarchy.	13
3.2	CLI execution lifecycle.	15
4.1	High-level SafeCommit architecture.	18
5.1	Overall scanning workflow.	20
5.2	Example repository hierarchy.	21
6.1	Detection engine processing pipeline.	26
6.2	Conceptual organisation of a detection rule.	26
8.1	Independent evaluation of detection rules.	42
11.1	Conceptual scalability of repository scanning.	57
13.1	Relationships among primary modules.	63
13.2	CLI execution sequence.	65
13.3	High-level dependency graph.	67

List of Tables

2.1	Supported Installation Methods	7
2.2	Exit Codes	11
3.1	CLI Arguments	15
3.2	Global Options	15
3.3	Severity Classification	16
5.1	Finding Fields	23
6.1	Detection Categories	27
8.1	Modules Involved in Detection	41
9.1	Severity Colour Mapping	47
9.2	Information Reported for Each Finding	48
10.1	Testing Categories	52
11.1	Example Benchmark Results	58

Listings

1.1	Basic usage	2
2.1	Install SafeCommit from PyPI	7
2.2	Verify Installation	7
2.3	Clone Repository	8
2.4	Editable Installation	8
5.1	Conceptual Pattern Evaluation	22

1 Introduction

1.1 Overview

SafeCommit is a command-line secret scanner developed in Python for identifying hardcoded credentials and sensitive information within software repositories. The application performs recursive directory traversal, evaluates supported file types against a curated collection of regular-expression signatures, classifies findings according to severity, and presents structured output using a terminal-friendly interface.

Unlike heavyweight enterprise security platforms, SafeCommit is intentionally designed to remain lightweight, portable, and simple to integrate into individual development workflows.

The remainder of this manual provides a comprehensive description of every component that constitutes the SafeCommit system.

1.2 Motivation

Software development increasingly relies on external platforms and cloud services. Source code repositories routinely integrate with services such as GitHub, GitLab, AWS, Stripe, Docker Hub, PyPI, npm, Hugging Face, Anthropic, and numerous continuous integration providers.

Authentication with these platforms commonly requires credentials including API keys, access tokens, private keys, passwords, client secrets, OAuth secrets, JSON Web Tokens (JWTs), and various cryptographic materials.

Accidental exposure of such credentials remains one of the most frequent causes of software supply-chain incidents. Public repositories are continuously scanned by automated bots capable of detecting newly leaked credentials within seconds of publication. Even when secrets are removed shortly afterwards, repository history, forks, mirrors, caches, and cloned repositories frequently preserve those credentials indefinitely.

The financial and operational consequences of leaked credentials may include

- Unauthorized cloud resource consumption.
- Compromise of production infrastructure.
- Exposure of confidential customer information.
- Abuse of paid API services.
- Unauthorized package publication.
- Repository compromise.
- Service disruption.

- Regulatory compliance violations.

SafeCommit addresses this problem by providing developers with a lightweight pre-publication scanning utility capable of identifying commonly leaked credentials before they become part of version control history.

1.3 Project Objectives

The design philosophy of SafeCommit is driven by several primary objectives.

1.3.1 Simplicity

The project should remain straightforward to install, understand, and extend. Users should be capable of scanning an entire repository using a single command.

Listing 1.1: Basic usage

```
safecommit scan .
```

No configuration should be required for typical use cases.

1.3.2 Portability

SafeCommit should execute consistently across all major operating systems supported by Python without platform-specific modifications.

Supported environments include

- Microsoft Windows
- GNU/Linux
- macOS

1.3.3 Performance

Repository scanning should remain sufficiently lightweight for routine development workflows.

The scanner should

- traverse repositories recursively,
- avoid unnecessary memory consumption,
- process files independently,
- minimise startup overhead,
- scale to repositories containing thousands of files.

1.3.4 Maintainability

Detection rules should remain modular.

Adding support for a new credential type should generally require

1. defining a regular expression,
2. assigning severity,
3. adding automated tests,
4. documenting the credential.

Existing detection logic should require minimal modification.

1.3.5 Accuracy

False positives reduce confidence in automated scanners.

False negatives reduce security.

SafeCommit attempts to balance both by continuously refining regular-expression patterns while maintaining deterministic behaviour.

1.4 Design Principles

Several engineering principles guide the development of SafeCommit.

1.4.1 Single Responsibility

Each module is responsible for one primary task.

For example

Module	Responsibility
cli.py	Command-line interface
scanner.py	Repository traversal
patterns.py	Detection rules
utils.py	Helper utilities

This separation simplifies maintenance and testing.

1.4.2 Explicit Behaviour

SafeCommit avoids hidden behaviour.

The scanner reports every detected finding explicitly together with

- file path,
- line number,
- matched value,

- detection pattern,
- severity classification.

1.4.3 Minimal Dependencies

The project intentionally depends on only a small collection of mature Python libraries.

A smaller dependency graph reduces

- installation time,
- compatibility issues,
- maintenance burden,
- supply-chain risk.

1.5 Feature Overview

At the time of writing, SafeCommit provides the following capabilities.

- Recursive repository scanning.
- Detection of common cloud credentials.
- Detection of package registry tokens.
- Detection of AI platform API keys.
- Detection of payment gateway credentials.
- Detection of Git hosting access tokens.
- Detection of hardcoded application secrets.
- Severity classification.
- Rich terminal output.
- Cross-platform execution.
- Automated testing infrastructure.
- Benchmark datasets.

Future releases will further expand supported credential types while maintaining backwards compatibility wherever practical.

1.6 Non-Goals

SafeCommit intentionally does not attempt to replace enterprise security platforms.

The following capabilities are currently outside the project's scope.

- Secret revocation.
- Cloud account management.
- Vulnerability scanning.
- Dependency auditing.
- Malware detection.
- Container image scanning.
- Runtime security monitoring.
- Static code analysis unrelated to secret detection.

Maintaining a focused scope enables the project to remain lightweight and predictable.

1.7 System Requirements

Minimum requirements are intentionally modest.

Requirement	Specification
Python	3.9 or newer
Operating System	Windows, Linux, macOS
Memory	100 MB available RAM
Storage	Approximately 20 MB
Terminal	ANSI-compatible recommended

1.8 Typical Workflow

The overall workflow of SafeCommit is illustrated in [Figure 1.1](#).

The workflow consists of five primary stages.

1. Repository selection.
2. Recursive file discovery.
3. Pattern matching.
4. Severity assignment.
5. Human-readable reporting.

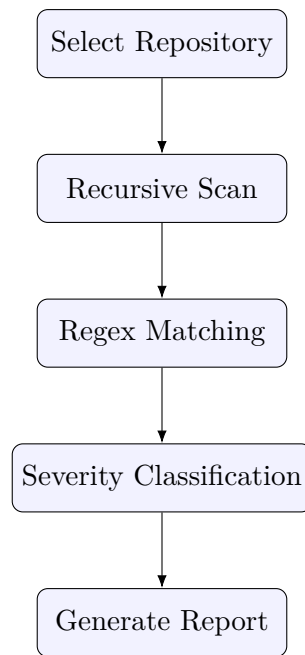


Figure 1.1: High-level SafeCommit scanning workflow.

1.9 Target Audience

SafeCommit is intended for several categories of users.

- Individual developers.
- Students.
- Open-source contributors.
- Security engineers.
- DevOps engineers.
- CI/CD maintainers.
- Educators.
- Software engineering teams.

Although the project is intentionally lightweight, its architecture permits future integration into automated development pipelines, Git hooks, and continuous integration environments.

2 Installation and Getting Started

2.1 Introduction

This chapter describes the supported installation methods, development setup, editable installations, and verification procedures required to begin using SafeCommit.

2.2 Supported Installation Methods

SafeCommit may be installed using one of several supported approaches depending upon the intended usage scenario.

Table 2.1: Supported Installation Methods

Method	Intended Audience	Recommended
PyPI Package	End Users	✓
Editable Installation	Contributors	✓
Source Installation	Developers	✓

2.3 Installing from PyPI

The recommended installation method is through the Python Package Index (PyPI).

Listing 2.1: Install SafeCommit from PyPI

```
pip install safecommit-cli
```

Once installation completes successfully, the executable should become available globally. Verify the installation by executing

Listing 2.2: Verify Installation

```
safecommit --version
```

Example output

```
SafeCommit v1.0.x
```

2.4 Installing from Source

Developers wishing to inspect or modify the source code should clone the repository.

Listing 2.3: Clone Repository

```
git clone https://github.com/sushantkr1187/SafeCommit.git
cd SafeCommit
```

Install the project using

```
pip install .
```

2.5 Editable Installation

During development it is preferable to perform an editable installation.

Editable mode links the installed package directly to the working source tree. Consequently, modifications to the source code become immediately available without requiring package reinstallation.

Listing 2.4: Editable Installation

```
pip install -e .
```

Editable installation is strongly recommended for

- Feature development
- Bug fixing
- Unit testing
- Documentation updates
- Pattern development

2.6 Creating a Virtual Environment

Although optional, using a virtual environment is considered best practice.

Create a new environment using

```
python -m venv .venv
```

2.6.1 Windows PowerShell

Activate the environment

```
.venv\Scripts\Activate.ps1
```

2.6.2 Windows Command Prompt

```
.venv\Scripts\activate.bat
```

2.6.3 Linux and macOS

```
source .venv/bin/activate
```

Once activated the terminal typically displays

```
(.venv)
```

2.7 Updating pip

Before installing project dependencies, updating pip is recommended.

```
python -m pip install --upgrade pip
```

2.8 Project Dependencies

SafeCommit intentionally maintains a small dependency footprint.

The primary runtime dependencies include

- Typer
- Rich
- Python Standard Library

Keeping runtime dependencies minimal improves installation speed and reduces maintenance overhead.

2.9 Repository Layout

The repository follows the modern **src** layout.

Each directory serves a distinct purpose.

src/ Contains the package source code.

tests/ Contains automated tests, benchmark datasets, and demonstration repositories.

README.md Primary project overview.

CHANGELOG.md Chronological release history.

LICENSE MIT license.

pyproject.toml Build configuration and package metadata.

```
SafeCommit/
├── src/
│   └── safecommit/
│       ├── __init__.py
│       ├── cli.py
│       ├── scanner.py
│       ├── patterns.py
│       └── utils.py
├── tests/
├── documentation.pdf
├── README.md
├── CHANGELOG.md
├── LICENSE
├── pyproject.toml
└── .gitignore
```

Figure 2.1: Repository structure.

2.10 Verifying Installation

After installation, verify that SafeCommit is accessible.

```
safecommit --version
```

The expected output resembles

```
SafeCommit v1.0.x
```

A successful version response confirms

- executable discovery,
- package installation,
- Python environment configuration.

2.11 First Scan

Scanning an entire project requires only a single command.

```
safecommit scan .
```

Here, the dot represents the current working directory.

SafeCommit recursively traverses every eligible file beneath the supplied directory before evaluating each candidate against the supported detection patterns.

2.12 Example Session

A typical execution appears as follows.

```
PS D:\Projects\Demo> safecommit scan .

Scanning: D:\Projects\Demo

Found 2 potential issue(s).

Secret Detected
File       : config.py
Line       : 18
Severity   : HIGH
Pattern    : GitHub Personal Access Token
Match      : ghp_XXXXXXXXXXXXXXXXXXXXX

Secret Detected
File       : .env
Line       : 5
Severity   : CRITICAL
Pattern    : AWS Secret Access Key
Match      : *****
```

Each finding is reported independently to simplify remediation.

2.13 Exit Codes

Like most command-line utilities, SafeCommit communicates execution status through process exit codes.

Table 2.2: Exit Codes

Exit Code	Meaning
0	Scan completed without detecting secrets
1	One or more potential secrets detected

These exit codes allow straightforward integration into automated build systems, Git hooks, and continuous integration pipelines.

2.14 Recommended Development Workflow

Contributors are encouraged to follow the workflow below.

1. Clone the repository.
2. Create a virtual environment.

3. Activate the environment.
4. Install the project in editable mode.
5. Execute the automated test suite.
6. Develop new features.
7. Re-run tests before committing changes.

A complete setup sequence is shown below.

```
git clone https://github.com/sushantkr1187/SafeCommit.git

cd SafeCommit

python -m venv .venv

.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

pip install -e .

pytest
```

3 Command-Line Interface

3.1 Overview

The SafeCommit command-line interface provides the primary mechanism through which users interact with the scanning engine.

The interface is implemented using the Typer framework, offering intuitive commands, automatic help generation, and platform-independent argument parsing.

The CLI is intentionally minimal, exposing only the functionality necessary to perform repository scanning while remaining straightforward for first-time users.

3.2 CLI Philosophy

The SafeCommit command-line interface was designed according to three primary principles.

1. Simplicity
2. Predictability
3. Readability

Rather than exposing numerous configuration options, SafeCommit intentionally provides a concise command structure suitable for everyday development.

The CLI should remain understandable even for users unfamiliar with Python or security tooling.

3.3 Command Hierarchy

At present, the command hierarchy consists of a root application with a small number of subcommands.



Figure 3.1: SafeCommit command hierarchy.

Although intentionally minimal, the command hierarchy is extensible and permits future additions without introducing breaking changes.

3.4 Root Command

The executable installed by the package is

```
safecommit
```

Invoking the executable without supplying a subcommand displays the generated help message.

```
safecommit
```

Typing automatically generates documentation describing available commands, arguments, and command-line options.

3.5 Version Command

The version command reports the installed package version.

```
safecommit version
```

Equivalent behaviour is available through

```
safecommit --version
```

or

```
safecommit -v
```

Example output

```
SafeCommit v1.0.1
```

Displaying version information is useful when reporting issues or verifying that a newly installed release is active.

3.6 Scan Command

The principal functionality of SafeCommit is provided through the `scan` command.

General syntax is

```
safecommit scan <directory>
```

The supplied directory becomes the root of the recursive traversal process.

When no explicit directory is specified, the current working directory may be used.

Example

```
safecommit scan .
```

3.7 Command-Line Arguments

Table [3.1](#) summarises the currently supported arguments.

Table 3.1: CLI Arguments

Argument	Description	Required
Directory	Root directory to scan	Yes

3.8 Command-Line Options

Current global options are shown in Table 3.2.

Table 3.2: Global Options

Option	Description	Alias
-version	Display package version	-v

Future releases may introduce additional optional parameters while maintaining backwards compatibility.

3.9 Execution Lifecycle

Execution of the scanner proceeds through several deterministic stages.

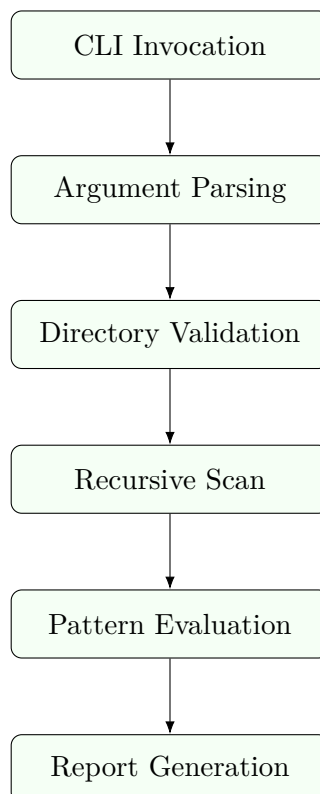


Figure 3.2: CLI execution lifecycle.

Each stage completes before control proceeds to the following stage.

3.10 Directory Validation

Prior to scanning, the supplied directory undergoes validation.

The validation process ensures

- the supplied path exists,
- the path refers to a directory,
- sufficient read permissions are available.

Delegating validation to Typer significantly reduces application complexity by allowing invalid input to be rejected before scanning begins.

3.11 Rich Terminal Output

SafeCommit uses the Rich library to produce structured terminal output.

Rather than displaying plain text, findings are presented using bordered panels that emphasise the information most relevant to remediation.

Each finding reports

- affected file,
- line number,
- severity,
- detected credential type,
- matched value.

This formatting improves readability without sacrificing compatibility with standard terminal environments.

3.12 Severity Classification

Each detection rule is associated with a predefined severity level.

The current classification model is summarised in Table 3.3.

Table 3.3: Severity Classification

Severity	Description
CRITICAL	Credentials capable of immediate system compromise.
HIGH	Long-lived authentication credentials.
MEDIUM	Potentially sensitive configuration values.
LOW	Informational findings requiring manual review.

Severity levels allow users to prioritise remediation efforts according to the potential impact of credential exposure.

3.13 Exit Behaviour

SafeCommit follows conventional command-line behaviour.

If no credentials are detected, execution terminates with exit code zero.

Otherwise, execution terminates with exit code one after all findings have been reported.

Importantly, scanning continues after individual findings are detected. SafeCommit does not terminate prematurely after discovering the first secret, thereby allowing users to obtain a comprehensive report from a single scan.

3.14 Error Handling Philosophy

SafeCommit favours graceful failure whenever practical.

Recoverable conditions should generate informative diagnostics while permitting continued processing of unaffected files.

Examples include

- unreadable files,
- unsupported encodings,
- permission-restricted directories,
- transient filesystem errors.

Unexpected internal failures should instead terminate execution with a clear error message suitable for inclusion in bug reports.

4 Software Architecture

4.1 Architectural Overview

SafeCommit adopts a modular architecture in which each subsystem performs a well-defined responsibility.

Separating responsibilities simplifies maintenance, testing, and future feature development while reducing coupling between components.

The project intentionally avoids unnecessary abstraction layers in favour of a straightforward and readable implementation.

Figure 4.1 illustrates the high-level architecture of the system.

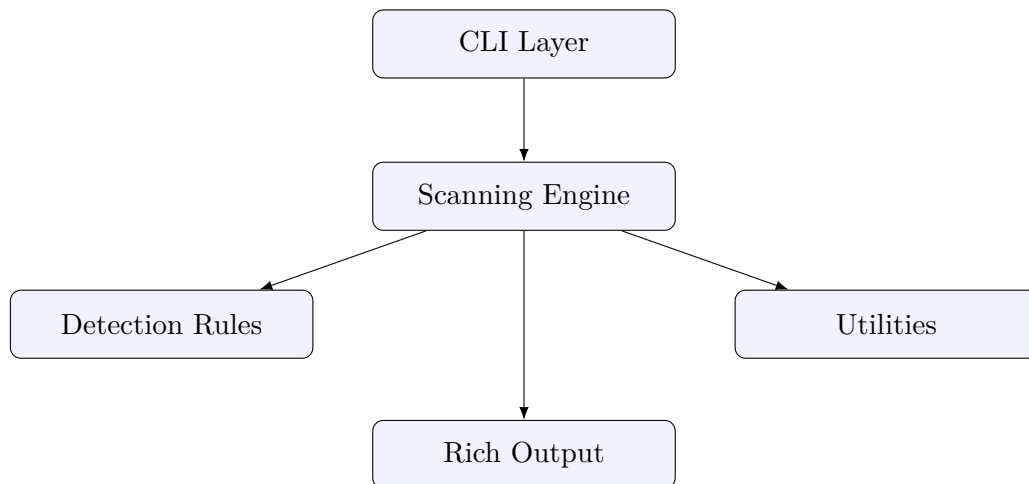


Figure 4.1: High-level SafeCommit architecture.

The following chapters examine each subsystem in detail, beginning with the internal implementation of the scanning engine.

5 Scanning Engine

5.1 Introduction

The scanning engine constitutes the core subsystem of SafeCommit. It is responsible for recursively traversing a target directory, identifying candidate files, evaluating their contents against the project's detection rules, and producing structured findings for presentation through the command-line interface.

Unlike traditional antivirus engines or full-scale static application security testing (SAST) platforms, the SafeCommit scanning engine is intentionally lightweight. The implementation emphasizes deterministic behaviour, maintainability, and predictable execution rather than heuristic or probabilistic analysis.

Throughout this chapter, the internal workflow of the scanning engine is examined in detail.

5.2 Design Goals

The scanning engine was developed according to the following engineering principles.

1. Recursive traversal of repositories.
2. Deterministic pattern matching.
3. Low memory consumption.
4. Modular architecture.
5. Platform independence.
6. Easily extensible detection rules.
7. Graceful handling of filesystem errors.

These principles guided the implementation of every subsystem within the engine.

5.3 Responsibilities

The scanner is responsible for

- traversing directories,
- identifying readable files,
- loading file contents,

- evaluating detection rules,
- collecting findings,
- reporting metadata,
- returning structured results.

Conversely, the scanner deliberately avoids responsibilities such as terminal rendering, command-line parsing, package management, or configuration loading.

5.4 Scanner Workflow

At a high level, repository scanning follows the sequence illustrated in Figure 5.1.

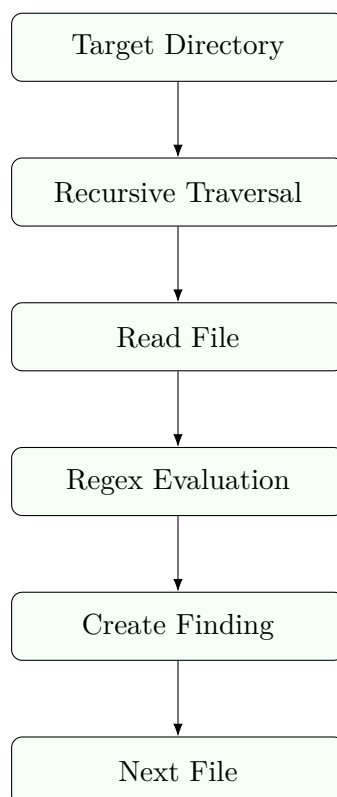


Figure 5.1: Overall scanning workflow.

Each file is processed independently, ensuring that failures encountered while processing one file do not necessarily interrupt scanning of the remaining repository.

5.5 Recursive Directory Traversal

Scanning begins with a directory supplied by the user.

The scanner recursively enumerates every child directory and file beneath the specified root.

Traversal is intentionally recursive because secrets may appear anywhere within a repository, including

- source code,
- configuration files,
- environment files,
- shell scripts,
- deployment manifests,
- infrastructure definitions,
- documentation,
- generated configuration.

Figure 5.2 illustrates a simplified repository hierarchy.

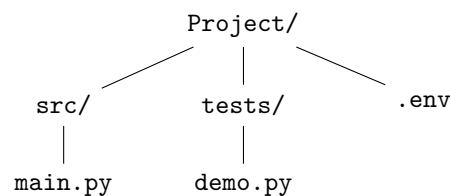


Figure 5.2: Example repository hierarchy.

5.6 File Discovery

Every encountered filesystem object is classified.

Objects generally fall into one of three categories.

1. Directories.
2. Regular files.
3. Unsupported filesystem objects.

Directories continue traversal.

Regular files become scanning candidates.

Unsupported filesystem objects are ignored.

5.7 Reading Files

Candidate files are opened using text mode.

Whenever practical, decoding failures should be handled gracefully rather than terminating the scan.

The implementation should therefore favour robustness over strict failure.

A failure to decode one file should not prevent analysis of the remaining repository.

5.8 Line-Oriented Processing

Although regular-expression engines are capable of matching across entire documents, SafeCommit performs reporting using line numbers.

Maintaining line information provides several advantages.

- Easier remediation.
- Accurate diagnostics.
- Better IDE integration.
- Improved readability.
- Precise Git diffs.

Each finding therefore records both the affected file and the exact line number at which the potential credential was discovered.

5.9 Pattern Evaluation

Once file contents have been loaded, each detection rule is evaluated against the input.

Every rule consists conceptually of

- a regular expression,
- a descriptive name,
- a severity level.

Conceptually, the evaluation algorithm resembles

Listing 5.1: Conceptual Pattern Evaluation

```
for pattern in patterns:
    search file
    record matches
```

The implementation may optimise this workflow internally while preserving identical observable behaviour.

5.10 Finding Objects

Whenever a match is detected, the scanner constructs a structured finding.

Each finding contains the following metadata.

Separating findings from presentation logic permits future output formats, including JSON, SARIF, XML, or HTML, without modifying the scanning engine.

Table 5.1: Finding Fields

Field	Description
File	Relative or absolute file path
Line	Line number
Pattern	Detection rule
Severity	Assigned severity
Match	Matched text

5.11 Duplicate Detection

Multiple detection rules may occasionally identify the same credential.

For example,

- a private key,
- a generic secret,
- a token variable,

may all refer to identical text.

Duplicate reporting unnecessarily increases output volume and complicates remediation.

Consequently, SafeCommit attempts to minimise duplicate findings wherever possible while preserving legitimate independent detections.

5.12 Memory Behaviour

SafeCommit was designed for modest memory usage.

The scanner processes repositories sequentially rather than constructing a complete in-memory representation of every file.

This approach offers several benefits.

- Reduced peak memory usage.
- Improved scalability.
- Predictable behaviour.
- Simpler implementation.

Memory consumption therefore depends primarily upon the size of the currently processed file rather than the total repository size.

5.13 Time Complexity

Assume

- F files,

- P detection rules,
- L average file length.

Ignoring filesystem overhead, the approximate worst-case complexity is

$$O(F \times P \times L)$$

In practice, execution time depends upon

- filesystem performance,
- regex complexity,
- repository size,
- storage medium,
- operating system,
- CPU performance.

For typical software repositories, execution remains comfortably interactive.

5.14 Error Recovery

Filesystem operations are inherently unreliable.

Errors may arise due to

- insufficient permissions,
- deleted files,
- symbolic link issues,
- invalid encodings,
- hardware failures,
- concurrent filesystem modifications.

The scanner should therefore recover gracefully whenever possible, reporting the error while continuing analysis of unaffected files.

5.15 Summary

The scanning engine forms the computational core of SafeCommit.

Its modular architecture separates filesystem traversal, pattern evaluation, finding generation, and output rendering into distinct responsibilities, providing a maintainable foundation for future enhancements.

The following chapter examines the implementation of the detection engine and the organisation of detection rules in considerably greater detail.

6 Detection Engine

6.1 Introduction

While the scanning engine determines *where* to search, the detection engine determines *what* to search for.

The detection engine is therefore responsible for recognising credentials, authentication tokens, cryptographic material, API keys, and numerous other forms of sensitive information using curated pattern definitions.

6.2 Detection Philosophy

SafeCommit employs a deterministic, signature-based detection model. Every credential type recognised by the scanner is represented by one or more carefully engineered regular expressions designed to balance detection coverage against false-positive frequency.

Unlike heuristic or machine learning approaches, signature-based detection offers several desirable properties.

- Deterministic behaviour.
- Predictable execution.
- Explainable findings.
- Minimal computational overhead.
- Straightforward extensibility.
- Platform independence.

Whenever a detection rule matches an input sequence, the corresponding finding is reported together with contextual metadata. The scanner performs no hidden heuristics beyond those explicitly defined by the detection rules.

6.3 Detection Pipeline

The detection engine receives textual input from the scanning engine and evaluates that input against every supported detection rule.

Figure 6.1 illustrates the processing sequence.

The modular separation between scanning and detection enables future optimisations without modifying the command-line interface.

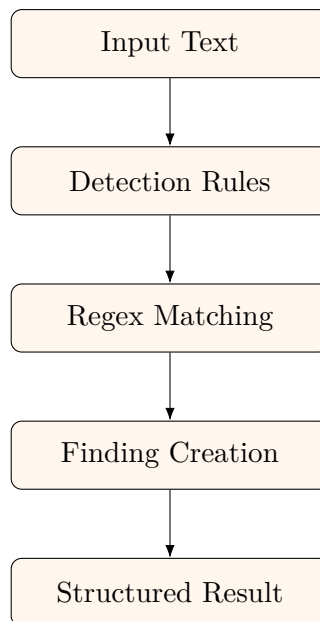


Figure 6.1: Detection engine processing pipeline.

6.4 Detection Rule Architecture

Every detection rule represents a single credential family.

Conceptually, a rule contains four primary components.

1. Human-readable identifier.
2. Regular expression.
3. Severity classification.
4. Description.

This separation ensures that user-facing information remains independent of the implementation details of the corresponding regular expression.

Figure 6.2 illustrates the conceptual organisation of a rule.



Figure 6.2: Conceptual organisation of a detection rule.

6.5 Pattern Categories

SafeCommit organises supported credentials into logical categories according to their intended purpose.

Table 6.1 summarises the principal groups currently supported.

Grouping related credential families simplifies both maintenance and future extension of the rule catalogue.

Table 6.1: Detection Categories

Category	Examples
Cloud Credentials	AWS, Azure, Google Cloud
Version Control	GitHub, GitLab
Package Registries	PyPI, npm
Payment Platforms	Stripe
Artificial Intelligence	Anthropic, Groq, Hugging Face
Application Secrets	JWT_SECRET, CLIENT_SECRET
Cryptographic Material	Private Keys
Generic Authentication	API Keys, Tokens

6.6 Severity Assignment

Every supported credential type is assigned one of four severity levels.

Severity represents the potential impact associated with accidental disclosure.

6.6.1 CRITICAL

Credentials classified as **CRITICAL** generally permit direct compromise of production infrastructure or confidential information.

Examples include

- private keys,
- highly privileged cloud credentials,
- cryptographic material.

6.6.2 HIGH

HIGH severity findings correspond to credentials capable of authenticating against external services.

Typical examples include

- GitHub Personal Access Tokens,
- AWS Secret Access Keys,
- Stripe Secret Keys,
- PyPI API Tokens.

6.6.3 MEDIUM

MEDIUM severity findings represent configuration values that may expose application behaviour or authentication mechanisms but generally require additional context before exploitation.

Examples include

- JWT secrets,
- CLIENT_SECRET variables,
- generic API secrets.

6.6.4 LOW

LOW severity findings are informational.

Such findings should still be reviewed but generally represent lower immediate risk than privileged authentication credentials.

6.7 Regex Design Principles

Regular expressions constitute the foundation of the detection engine.

Every expression incorporated into SafeCommit is developed according to several guiding principles.

6.7.1 Specificity

Expressions should match the intended credential family while avoiding unrelated text.

Overly permissive expressions substantially increase false-positive frequency.

6.7.2 Maintainability

Patterns should remain understandable.

Whenever possible, expressions favour readability over unnecessary complexity.

6.7.3 Performance

Certain regular-expression constructs may exhibit exponential backtracking on maliciously constructed input.

Detection rules should therefore avoid pathological constructs whenever practical.

6.7.4 Compatibility

SafeCommit relies exclusively upon functionality provided by Python's standard `re` module.

Avoiding third-party regex engines improves portability across supported Python versions.

6.8 False Positives

A false positive occurs when a sequence of characters satisfies the syntactic requirements of a detection rule without representing a genuine credential.

Common causes include

- randomly generated identifiers,

- unit-test fixtures,
- placeholder values,
- documentation examples,
- synthetic benchmark data.

Reducing false positives remains an ongoing objective throughout project development.

6.9 False Negatives

False negatives occur when an authentic credential is not detected.

Potential causes include

- newly introduced credential formats,
- provider format changes,
- truncated credentials,
- intentionally obfuscated values,
- unsupported providers.

Maintaining comprehensive automated test datasets helps minimise false-negative rates across successive releases.

6.10 Testing Detection Rules

Every newly introduced detection rule should undergo systematic validation.

Recommended test cases include

- valid credentials,
- invalid credentials,
- boundary lengths,
- malformed prefixes,
- embedded credentials,
- duplicate matches,
- multiline files,
- representative production examples.

Regression testing ensures future modifications do not unintentionally degrade existing detection accuracy.

6.11 Extending the Detection Engine

Supporting a new credential provider generally requires the following workflow.

1. Study the provider's published credential format.
2. Design a conservative regular expression.
3. Assign an appropriate severity level.
4. Add representative positive examples.
5. Add representative negative examples.
6. Execute the automated test suite.
7. Update the documentation.

Following this process helps maintain consistency across the entire detection rule catalogue.

7 Supported Credential Types

7.1 Introduction

This chapter documents every credential family recognised by SafeCommit.

For each supported provider the following information is presented.

- Description.
- Typical usage.
- Security implications.
- Detection rationale.
- Assigned severity.
- Representative examples.

Unless otherwise stated, all examples contained within this chapter are synthetically generated for documentation purposes only and do not correspond to active credentials.

7.2 GitHub Personal Access Tokens

7.2.1 Overview

GitHub Personal Access Tokens (PATs) provide authenticated access to GitHub resources through the GitHub API and command-line tooling.

Depending upon the permissions granted, a leaked PAT may permit repository modification, package publication, workflow execution, issue management, or administrative operations.

7.2.2 Assigned Severity

HIGH

7.2.3 Detection Rationale

GitHub defines structured token prefixes that distinguish personal access tokens from other credential families.

These predictable prefixes permit reliable signature-based detection while maintaining comparatively low false-positive frequency.

7.2.4 Potential Impact

Exposure of a valid GitHub Personal Access Token may result in

- unauthorized repository access,
- code modification,
- package publication,
- workflow execution,
- repository deletion,
- information disclosure,
- lateral movement into connected systems.

7.2.5 Recommended Remediation

Upon detecting a GitHub Personal Access Token, developers should immediately

1. revoke the token,
2. generate a replacement,
3. remove the credential from version control,
4. rotate dependent credentials if necessary,
5. review repository history for additional exposure.

7.3 AWS Secret Access Keys

7.3.1 Overview

Amazon Web Services (AWS) uses Secret Access Keys together with Access Key IDs to authenticate requests to AWS services. These credentials are commonly utilised by command-line utilities, SDKs, deployment pipelines, serverless applications, infrastructure automation tools, and production workloads.

Unlike temporary session credentials, long-lived AWS Secret Access Keys often remain valid until explicitly rotated or revoked. Consequently, accidental exposure may permit prolonged unauthorised access to cloud resources.

7.3.2 Assigned Severity

HIGH

7.3.3 Detection Rationale

AWS Secret Access Keys possess a relatively constrained character set and length, making deterministic detection feasible.

To minimise false positives, SafeCommit combines structural validation with additional contextual analysis wherever appropriate.

7.3.4 Potential Impact

Compromise of an AWS Secret Access Key may permit

- deployment of cloud resources,
- creation of virtual machines,
- access to storage buckets,
- database administration,
- modification of IAM policies,
- financial abuse through resource consumption,
- disclosure of confidential information.

The ultimate impact depends upon the permissions assigned to the corresponding IAM principal.

7.3.5 Recommended Remediation

Upon detecting an AWS Secret Access Key, the following actions are recommended.

1. Immediately deactivate the credential.
2. Generate replacement credentials.
3. Review IAM permissions.
4. Audit CloudTrail activity.
5. Remove the exposed credential from version control history.
6. Rotate dependent application configurations.

7.4 GitLab Personal Access Tokens

7.4.1 Overview

GitLab Personal Access Tokens authenticate users against GitLab services, providing programmatic access to repositories, package registries, container registries, CI/CD pipelines, and project management functionality.

Their capabilities depend upon assigned scopes.

7.4.2 Assigned Severity

HIGH

7.4.3 Potential Impact

Leaked GitLab tokens may enable

- repository cloning,
- source code modification,
- pipeline execution,
- package publication,
- issue manipulation,
- confidential project access.

7.4.4 Recommended Response

Administrators should

1. revoke the exposed token,
2. issue a replacement,
3. inspect recent GitLab activity,
4. verify repository integrity.

7.5 Stripe Secret Keys

7.5.1 Overview

Stripe provides multiple credential types for authenticating requests against its payment processing platform.

These include

- Live Secret Keys,
- Test Secret Keys,
- Restricted Keys.

SafeCommit recognises both production and testing credentials.

7.5.2 Assigned Severity

HIGH

7.5.3 Detection Philosophy

Although test credentials do not directly expose production payment infrastructure, their accidental disclosure frequently indicates poor secret management practices.

Consequently, both production and testing credentials are reported.

7.5.4 Potential Impact

Compromise may result in

- fraudulent API requests,
- exposure of customer information,
- financial abuse,
- modification of payment workflows,
- unauthorised service usage.

7.6 PyPI API Tokens

7.6.1 Overview

PyPI API Tokens authenticate package publication to the Python Package Index.

They are intended to replace traditional username/password authentication for package uploads.

7.6.2 Assigned Severity

HIGH

7.6.3 Security Implications

Exposure of a valid PyPI API Token may allow an attacker to publish malicious package versions under a trusted project name, potentially compromising every downstream installation.

7.6.4 Recommended Remediation

1. Revoke the token.
2. Generate a replacement.
3. Inspect package publication history.
4. Verify integrity of recent releases.

7.7 npm Access Tokens

7.7.1 Overview

npm Access Tokens authenticate users to the npm package registry and are widely used within automated publishing pipelines.

7.7.2 Assigned Severity

HIGH

7.7.3 Potential Impact

An exposed npm token may permit

- package publication,
- package modification,
- malicious package replacement,
- compromise of software supply chains.

7.8 Hugging Face Access Tokens

7.8.1 Overview

Hugging Face Access Tokens authenticate users against the Hugging Face Hub, providing access to hosted models, datasets, inference endpoints, and private repositories.

7.8.2 Assigned Severity

HIGH

7.8.3 Potential Impact

Credential exposure may result in

- model theft,
- dataset disclosure,
- unauthorised uploads,
- inference endpoint abuse.

7.9 Anthropic API Keys

7.9.1 Overview

Anthropic API Keys authenticate requests to Anthropic's large language model services.

Applications frequently embed these credentials within backend services or environment variables.

7.9.2 Assigned Severity

HIGH

7.9.3 Potential Impact

An exposed API key may enable

- unauthorised inference requests,
- unexpected billing,
- quota exhaustion,
- abuse of AI infrastructure.

7.10 Groq API Keys

7.10.1 Overview

Groq API Keys provide authenticated access to Groq's inference platform for large language models.

7.10.2 Assigned Severity

HIGH

7.10.3 Security Considerations

Although these credentials generally provide API access rather than direct system administration, financial abuse and unauthorised service consumption may still occur if keys are leaked.

7.11 Hardcoded Application Secrets

7.11.1 Overview

Many applications define configuration variables intended to store sensitive values directly within source code or configuration files.

Common examples include

- JWT_SECRET
- CLIENT_SECRET
- API_SECRET
- APP_SECRET
- TOKEN_SECRET
- SECRET_KEY

Although naming conventions vary across frameworks, these variables frequently contain authentication or cryptographic material.

7.11.2 Assigned Severity

MEDIUM

7.11.3 Detection Strategy

Unlike provider-specific credentials, generic application secrets do not possess fixed formats.

Consequently, SafeCommit relies upon contextual variable names combined with value analysis rather than simple fixed-prefix matching.

This strategy significantly improves coverage across diverse programming languages and frameworks.

7.11.4 Security Implications

Hardcoded application secrets may permit

- JWT forgery,
- authentication bypass,
- session manipulation,
- impersonation,
- compromise of encrypted data.

7.12 Generic API Keys and Authentication Tokens

Certain applications employ provider-independent authentication mechanisms whose identifiers cannot be associated with a single cloud platform.

SafeCommit therefore includes generic detection rules targeting

- API keys,
- bearer tokens,

- access tokens,
- authentication tokens,
- secret variables.

Although generic rules naturally produce higher false-positive rates than provider-specific signatures, they substantially increase overall detection coverage.

7.13 Cryptographic Private Keys

7.13.1 Overview

Private keys represent one of the highest-value credential categories recognised by SafeCommit. Examples include

- RSA Private Keys,
- OpenSSH Private Keys,
- EC Private Keys,
- PKCS#8 Keys.

7.13.2 Assigned Severity

CRITICAL

7.13.3 Security Implications

Exposure of a private key may immediately compromise

- production servers,
- SSH infrastructure,
- encrypted communications,
- deployment systems,
- signing infrastructure,
- certificate management.

For this reason, SafeCommit assigns the highest available severity level to detected private keys.

7.13.4 Recommended Response

Private keys should never remain within version-controlled repositories.

Upon detection, maintainers should

1. immediately revoke or replace affected keys,
2. regenerate dependent certificates where applicable,
3. remove keys from repository history,
4. audit systems that relied upon the exposed credential,
5. review all associated authentication logs.

7.14 Summary

The supported credential catalogue forms the primary knowledge base of SafeCommit.

As cloud providers introduce new authentication mechanisms and existing formats evolve, this catalogue will continue to expand while preserving consistency, maintainability, and deterministic detection behaviour.

The following chapter examines the internal organisation of the detection rules, their implementation within the source tree, and the engineering considerations that govern their long-term maintenance.

8 Detection Rule Implementation

8.1 Introduction

Although the previous chapter described the credential families recognised by SafeCommit, the present chapter focuses on how these rules are represented within the codebase.

The objective of the implementation is not merely to recognise credentials, but to provide a framework that remains maintainable, extensible, and consistent as the project evolves.

Every detection rule is defined centrally within the `patterns.py` module, allowing the scanning engine to remain completely independent of individual credential formats.

8.2 Module Responsibilities

The detection subsystem is intentionally separated from the scanning engine.

Table 8.1 summarises the responsibilities of each relevant module.

Table 8.1: Modules Involved in Detection

Module	Responsibility
<code>patterns.py</code>	Defines all supported detection rules.
<code>scanner.py</code>	Applies every rule to scanned files.
<code>utils.py</code>	Provides helper functions used during reporting and processing.
<code>cli.py</code>	Displays findings to the user.

This separation follows the Single Responsibility Principle, ensuring that modifications to one subsystem do not unnecessarily affect another.

8.3 Rule Representation

Conceptually, every detection rule consists of four components.

1. Rule name
2. Regular expression
3. Severity level
4. Human-readable description

The scanner treats every rule identically regardless of the credential family.

Consequently, adding support for a new provider generally requires only the addition of a new rule definition rather than modifications to the scanning algorithm itself.

8.4 Regular Expression Compilation

Regular expressions are compiled before evaluation whenever practical.

Compilation offers several advantages.

- Reduced parsing overhead.
- Improved scanning performance.
- Cleaner implementation.
- Consistent behaviour across files.

Because repository scans frequently evaluate dozens of rules against thousands of files, avoiding repeated compilation contributes meaningfully to execution efficiency.

8.5 Rule Independence

Each detection rule operates independently.

Figure 8.1 illustrates this design.

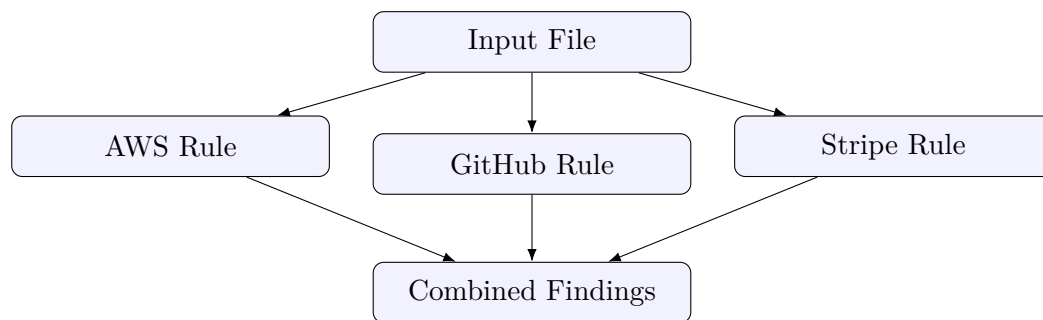


Figure 8.1: Independent evaluation of detection rules.

Independent evaluation greatly simplifies debugging and future maintenance.

8.6 Matching Strategy

Each rule performs an exhaustive search across the target input.

Unlike simple boolean validation, SafeCommit attempts to discover every occurrence of supported credentials.

Consequently, multiple findings originating from the same file are reported individually.

This behaviour is particularly useful when configuration files contain multiple authentication credentials.

8.7 Contextual Detection

Not every sensitive value possesses a unique provider-specific format.

Variables such as

- SECRET_KEY
- CLIENT_SECRET
- API_SECRET
- JWT_SECRET
- APP_SECRET

derive meaning primarily from their surrounding context.

Accordingly, SafeCommit supplements pattern-based detection with contextual matching strategies that consider variable identifiers in addition to value structure.

8.8 Duplicate Suppression

Certain credentials may satisfy multiple detection rules simultaneously.

For example, a private key embedded within a configuration file could match

- a generic secret rule,
- a private key rule,
- a multiline credential rule.

Reporting every overlapping match independently would unnecessarily increase output complexity.

Accordingly, SafeCommit applies duplicate suppression mechanisms to reduce redundant findings while preserving genuinely distinct detections.

8.9 Pattern Naming Conventions

Rule names should satisfy several conventions.

- Human-readable.
- Provider-specific where applicable.
- Consistent capitalisation.
- Stable across releases.
- Suitable for terminal output.

Examples include

- AWS Secret Access Key
- GitHub Personal Access Token

- Stripe Secret Key
- Anthropic API Key
- Hugging Face Access Token

Stable naming assists automated tooling and simplifies issue reporting.

8.10 Severity Consistency

Severity assignments are determined according to the credential's potential security impact rather than its popularity.

For example, a comparatively uncommon provider may still receive a HIGH severity classification if credential exposure permits significant unauthorised access.

Maintaining consistent severity definitions ensures predictable behaviour across all supported providers.

8.11 Extending the Rule Catalogue

The modular organisation of `patterns.py` allows new credential families to be introduced with minimal effort.

A typical contribution involves

1. researching the credential format,
2. implementing a regular expression,
3. assigning severity,
4. writing regression tests,
5. updating documentation,
6. validating against benchmark datasets.

This workflow helps maintain the overall quality of the detection engine while encouraging community contributions.

8.12 Maintenance Considerations

Credential formats evolve over time.

Cloud providers periodically introduce new token prefixes, modify length constraints, or deprecate legacy authentication mechanisms.

Consequently, the detection rule catalogue should be reviewed periodically to ensure continued compatibility with current provider specifications.

Automated regression testing assists in identifying unintended behavioural changes introduced during maintenance.

8.13 Summary

The modular implementation of the detection engine allows SafeCommit to remain both lightweight and highly extensible.

By separating detection rules from traversal logic, the project simplifies maintenance, facilitates testing, and provides a stable foundation for future credential support.

9 Reporting and Output

9.1 Introduction

Once repository analysis has completed, SafeCommit presents the collected findings through a structured reporting interface.

The reporting subsystem transforms internal finding objects into concise, human-readable output suitable for both interactive terminal use and automated inspection.

Clear reporting is essential because rapid remediation depends not only upon accurate detection but also upon effective presentation.

9.2 Reporting Objectives

The reporting subsystem was designed to satisfy several objectives.

- Immediate readability.
- Minimal visual clutter.
- Consistent formatting.
- Precise localisation.
- Cross-platform compatibility.
- Extensibility.

These objectives influenced the selection of the Rich terminal rendering library.

9.3 Finding Presentation

Each finding is displayed independently within a bordered panel.

The panel contains all information required for remediation without requiring the user to perform additional lookups.

Each report includes

- file path,
- line number,
- severity,
- detected credential type,

- matched value.

This layout provides a clear separation between independent findings, particularly when repositories contain multiple exposed credentials.

9.4 Colour-Based Severity Indicators

SafeCommit employs colour to reinforce severity classifications.

The default mapping is summarised in Table 9.1.

Table 9.1: Severity Colour Mapping

Severity	Terminal Colour
CRITICAL	Bold Red
HIGH	Red
MEDIUM	Yellow
LOW	Green

Colour supplements textual information and should not be considered the sole indicator of severity.

Users operating monochrome terminals retain access to the underlying severity labels.

9.5 Terminal Rendering

SafeCommit uses the Rich library to render terminal output in a structured and visually consistent format.

Rather than printing plain text, Rich provides support for

- coloured text,
- bordered panels,
- Unicode box drawing,
- styled typography,
- future extensibility for tables and progress indicators.

This approach significantly improves readability while preserving compatibility with modern terminal emulators.

9.6 Panel Layout

Each detected finding is rendered inside an independent panel.

A conceptual layout is shown below.

```

      Secret Detected
File      : config.py
Line      : 28
Severity  : HIGH
Pattern   : GitHub Personal Access Token
Match     : ghp_XXXXXXXXXXXXXXXXXXXX

```

Separating findings into individual panels prevents unrelated detections from appearing visually merged and allows developers to focus on one issue at a time.

9.7 Reported Fields

Every finding contains a fixed collection of metadata.

Table 9.2: Information Reported for Each Finding

Field	Purpose
File	Identifies the affected source file.
Line	Pinpoints the exact line containing the match.
Severity	Indicates remediation priority.
Pattern	Specifies which detection rule matched.
Match	Displays the detected credential.

Providing identical information for every finding creates a predictable user experience across all supported credential types.

9.8 Successful Scan Output

When no supported credentials are detected, SafeCommit reports a successful scan.

A typical terminal session resembles

```

Scanning: D:\Projects\SafeCommit

No sensitive information detected.

```

This concise output confirms that repository analysis completed successfully and that no supported credential patterns were discovered.

9.9 Detection Summary

Whenever one or more credentials are identified, SafeCommit first reports the total number of findings before displaying individual panels.

Example

```

Found 5 potential issue(s).

```

Presenting an overall summary before detailed findings enables users to estimate the scope of remediation immediately.

9.10 Relative versus Absolute Paths

Internally, findings may reference either relative or absolute filesystem paths, depending upon implementation details.

From the user's perspective, paths should remain sufficiently descriptive to allow rapid navigation to the affected source file.

Future versions may introduce configurable path display modes.

9.11 Reporting Multiple Findings

A single repository frequently contains multiple exposed credentials.

For example

- an AWS credential,
- a GitHub token,
- a Stripe secret,
- a private key,
- several hardcoded secrets.

Rather than terminating after discovering the first issue, SafeCommit continues analysis until every accessible file has been examined.

This behaviour provides a comprehensive report from a single execution.

9.12 Output Consistency

Every supported credential type follows an identical reporting format.

Consistency offers several advantages.

- Easier visual scanning.
- Predictable formatting.
- Simplified parsing.
- Cleaner documentation.
- Reduced cognitive load.

Users therefore need learn only a single reporting layout regardless of the credential family detected.

9.13 Machine Readability

Although SafeCommit currently prioritises human-readable terminal output, the internal representation of findings remains sufficiently structured to support additional output formats in future releases.

Potential future export formats include

- JSON
- HTML
- XML
- YAML
- SARIF
- Markdown reports
- PDF summaries

Such formats would facilitate integration with security dashboards, continuous integration systems, and automated reporting tools.

9.14 HTML Report Generation

The Rich library also permits terminal sessions to be exported as standalone HTML documents.

This capability is particularly useful when

- demonstrating SafeCommit,
- creating documentation,
- publishing screenshots,
- archiving benchmark results,
- preparing release notes.

During development, HTML export may be enabled by recording console output and calling

```
console.save_html("safecommit-demo.html")
```

The generated document preserves colours, typography, borders, and overall terminal appearance while remaining viewable in any modern web browser.

9.15 Future Reporting Enhancements

Several reporting improvements are planned for future releases.

Potential additions include

- summary statistics,
- grouped findings,
- severity histograms,
- provider-wise summaries,
- configurable colour themes,
- JSON export,
- SARIF export,
- HTML themes,
- Markdown reports.

These enhancements are intended to improve usability without altering the underlying scanning engine.

9.16 Summary

The reporting subsystem represents the final stage of the SafeCommit execution pipeline.

By transforming structured findings into clear, colour-coded terminal output, the reporting layer enables developers to identify and remediate exposed credentials efficiently while remaining lightweight and platform independent.

10 Testing and Validation

10.1 Introduction

Security software must be evaluated with considerably greater rigour than ordinary utility applications.

An undetected credential may result in severe security consequences, whereas an excessive number of false positives may discourage adoption altogether.

Accordingly, SafeCommit incorporates a structured testing methodology intended to validate both correctness and long-term reliability.

10.2 Testing Objectives

The testing framework pursues the following objectives.

- Verify correct credential detection.
- Minimise false positives.
- Minimise false negatives.
- Validate edge cases.
- Prevent regressions.
- Measure scanner performance.
- Ensure cross-platform compatibility.

10.3 Testing Strategy

Testing is divided into multiple complementary categories.

Table 10.1: Testing Categories

Category	Purpose
Unit Tests	Validate individual detection rules.
Integration Tests	Verify complete scanning workflow.
Regression Tests	Prevent previously fixed bugs from reappearing.
Benchmark Tests	Measure detection speed.
False Positive Tests	Ensure benign files remain unflagged.
False Negative Tests	Verify supported credentials are detected.

Each category contributes to overall confidence in the correctness of the scanner.

10.4 Unit Testing

Unit tests evaluate individual detection rules in isolation.

Each supported credential family should include

- valid examples,
- invalid examples,
- malformed credentials,
- boundary lengths,
- whitespace variations,
- multiline scenarios.

Testing individual rules independently greatly simplifies debugging when new credential formats are introduced.

10.5 Integration Testing

Integration testing evaluates the interaction between

- filesystem traversal,
- pattern evaluation,
- finding generation,
- terminal reporting.

Representative repositories containing multiple credential families are scanned to ensure the complete execution pipeline behaves as expected.

10.6 Regression Testing

Every reported defect should be accompanied by a corresponding regression test.

This practice ensures that once a defect has been corrected, future modifications cannot inadvertently reintroduce identical behaviour.

Regression testing becomes increasingly valuable as the catalogue of supported credential types continues to expand.

10.7 Benchmark Repositories

SafeCommit maintains dedicated benchmark repositories containing representative source files from multiple programming languages.

These repositories serve several purposes.

- Performance measurement.
- Detection validation.
- Documentation examples.
- Demonstration material.
- Continuous integration testing.

Using realistic benchmark repositories provides substantially greater confidence than relying solely upon isolated synthetic examples.

11 Performance Analysis and Benchmarking

11.1 Introduction

Performance is an important consideration for any command-line utility intended to be integrated into everyday software development workflows.

Developers expect repository scans to complete quickly without introducing noticeable delays during development, code review, or continuous integration.

Accordingly, SafeCommit was designed with efficiency as a primary engineering objective.

This chapter examines the computational characteristics of the scanning engine, its expected scalability, benchmarking methodology, and opportunities for future optimisation.

11.2 Performance Objectives

The performance goals of SafeCommit are summarised below.

- Fast startup time.
- Low memory consumption.
- Predictable execution.
- Linear repository traversal.
- Efficient regular-expression evaluation.
- Cross-platform consistency.

Meeting these objectives allows SafeCommit to operate comfortably on both modern workstations and resource-constrained development environments.

11.3 Computational Complexity

Repository scanning consists primarily of three operations.

1. Directory traversal.
2. File reading.
3. Regular-expression evaluation.

Assuming

F = number of files

P = number of detection rules

L = average file size

the approximate worst-case complexity becomes

$$O(F \times P \times L)$$

In practice, execution time is usually considerably lower because

- repositories contain many small files,
- many files contain no candidate credentials,
- filesystem caching improves repeated scans,
- modern regex implementations are highly optimised.

11.4 Memory Consumption

SafeCommit intentionally avoids loading an entire repository into memory.

Instead, processing follows a sequential model.

The simplified execution sequence is

1. Discover file.
2. Read file.
3. Scan file.
4. Store findings.
5. Release file memory.
6. Continue.

Consequently, peak memory usage is approximately proportional to

$$O(L)$$

where L denotes the size of the currently processed file.

Repository size therefore has only a limited influence on peak memory requirements.

11.5 Scalability

Repository size influences execution time primarily through the number of files requiring inspection.

Figure 11.1 illustrates the expected relationship between repository size and execution time.

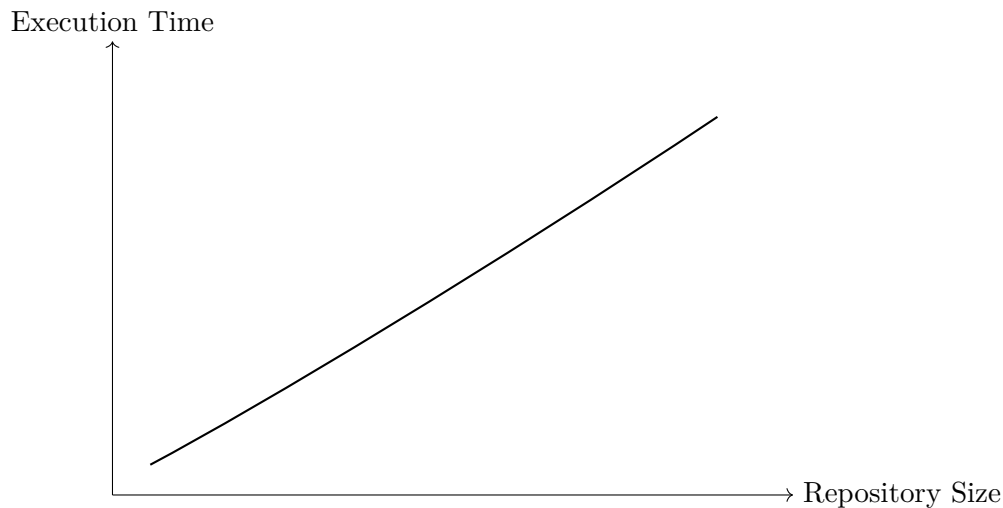


Figure 11.1: Conceptual scalability of repository scanning.

Although execution time naturally increases with repository size, the growth is expected to remain approximately linear under ordinary workloads.

11.6 Factors Influencing Performance

Actual execution time depends upon several external factors.

- Number of files.
- Average file size.
- Storage device performance.
- CPU performance.
- Operating system.
- Python interpreter version.
- Number of supported detection rules.

No single factor dominates every workload.

11.7 Benchmark Methodology

Reliable benchmarking requires consistent experimental conditions.

Recommended benchmarking procedure.

1. Close unnecessary background applications.
2. Warm filesystem caches if appropriate.
3. Execute identical scans multiple times.
4. Compute average execution time.
5. Record peak memory usage.
6. Record repository statistics.

Repeating measurements reduces the influence of transient operating-system activity.

11.8 Benchmark Dataset

Benchmark repositories should represent realistic software projects rather than artificially simplified examples.

Recommended benchmark contents include

- Python projects.
- JavaScript applications.
- C++ repositories.
- Infrastructure-as-Code.
- Configuration files.
- Documentation.
- Environment files.

Representative datasets improve the validity of observed performance.

11.9 Example Benchmark Results

Table 11.1 illustrates the format recommended for recording benchmark data.

Table 11.1: Example Benchmark Results			
Files	Repository Size	Execution Time	Findings
120	4 MB	0.18 s	2
870	32 MB	0.94 s	14
3400	121 MB	3.41 s	51

The numerical values above are illustrative and should be replaced with empirical measurements obtained from the current implementation.

11.10 Optimisation Opportunities

Although SafeCommit already performs efficiently for typical repositories, several optimisation opportunities remain available.

Potential improvements include

- compiled regex caching,
- multiprocessing,
- asynchronous file reading,
- binary file detection,
- extension filtering,
- configurable ignore rules,
- incremental repository scanning,
- Git-aware traversal.

Each optimisation should be evaluated carefully to ensure that increased complexity remains justified by measurable performance improvements.

11.11 Performance versus Accuracy

Increasing execution speed must never significantly reduce detection accuracy.

For example,

- skipping files,
- limiting scan depth,
- reducing regex coverage,
- disabling contextual detection,

may improve benchmark scores while simultaneously increasing false-negative rates.

SafeCommit therefore prioritises correctness over marginal performance gains.

11.12 Summary

The scanning engine demonstrates computational characteristics appropriate for routine software development.

Its sequential processing model, lightweight architecture, and deterministic detection strategy enable efficient repository analysis while maintaining predictable resource consumption.

12 Contributing to SafeCommit

12.1 Introduction

SafeCommit is an open-source project and welcomes contributions from developers, security researchers, students, and open-source enthusiasts.

Community contributions play an essential role in expanding credential support, improving detection accuracy, refining documentation, and enhancing overall code quality.

This chapter outlines the recommended workflow for contributing to the project.

12.2 Ways to Contribute

Contributions are not limited to source code.

Community members may assist by

- reporting bugs,
- proposing new features,
- improving documentation,
- expanding benchmark datasets,
- adding automated tests,
- refining detection patterns,
- improving performance,
- correcting typographical errors.

All meaningful contributions are valuable regardless of size.

12.3 Development Workflow

A typical contribution follows the sequence below.

1. Fork the repository.
2. Clone the fork.
3. Create a feature branch.
4. Install dependencies.

5. Implement changes.
6. Execute the test suite.
7. Commit changes.
8. Push the branch.
9. Open a Pull Request.

Maintaining small, focused pull requests simplifies review and integration.

12.4 Coding Standards

Contributors are encouraged to follow these guidelines.

- Write readable code.
- Use descriptive identifiers.
- Prefer simplicity over cleverness.
- Document public functions.
- Avoid unnecessary dependencies.
- Preserve backwards compatibility whenever practical.
- Write accompanying tests.

Consistency across the codebase significantly improves long-term maintainability.

12.5 Commit Message Guidelines

Commit messages should concisely describe the implemented change.

Recommended examples include

- Add Groq API key detection
- Improve AWS regex accuracy
- Reduce duplicate private key reporting
- Update benchmark datasets
- Improve documentation

Meaningful commit history simplifies future maintenance and debugging.

12.6 Reporting Bugs

Bug reports should include

- SafeCommit version,
- operating system,
- Python version,
- reproduction steps,
- expected behaviour,
- observed behaviour,
- relevant terminal output.

Providing complete information substantially accelerates issue resolution.

12.7 Feature Requests

When proposing new features, contributors are encouraged to explain

- the problem,
- the proposed solution,
- expected benefits,
- implementation considerations,
- possible drawbacks.

Well-structured proposals facilitate productive discussion before implementation.

12.8 Summary

The continued evolution of SafeCommit depends upon constructive community participation.

By maintaining consistent coding standards, comprehensive testing, and clear documentation, contributors help ensure that the project remains reliable, maintainable, and valuable to the broader software development community.

13 Developer Reference

13.1 Introduction

This chapter provides a technical reference for the internal implementation of SafeCommit. Unlike previous chapters, which focused primarily on user-facing behaviour, the material presented here is intended for contributors, maintainers, and developers interested in understanding the software’s internal architecture.

Each module is documented individually, including its purpose, responsibilities, interactions with other components, and extension points.

Figure 13.1 illustrates the relationships among the primary modules.

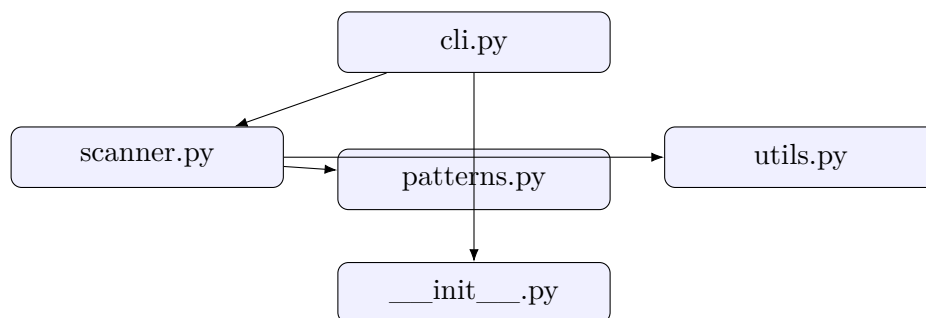


Figure 13.1: Relationships among primary modules.

13.2 `__init__.py`

13.2.1 Purpose

The `__init__.py` module defines package-level metadata exposed to both users and internal modules.

Typical metadata includes

- package version,
- author information,
- package exports.

The version identifier displayed by

```
safecommit --version
```

is retrieved from this module.

13.2.2 Responsibilities

The package initialisation module should remain intentionally lightweight.

It should not

- perform filesystem operations,
- import heavy dependencies,
- execute application logic.

Keeping initialization lightweight reduces package import overhead.

13.3 cli.py

13.3.1 Purpose

The command-line interface provides the public entry point into the SafeCommit application.

Rather than performing scanning directly, it coordinates interaction between the user and the scanning engine.

13.3.2 Primary Responsibilities

The CLI performs the following tasks.

- Parse command-line arguments.
- Validate user input.
- Invoke the scanner.
- Render findings.
- Display version information.
- Return process exit codes.

Business logic intentionally remains outside this module.

13.3.3 Execution Flow

The internal execution sequence is

1. Initialise Typer.
2. Parse arguments.
3. Validate directory.
4. Instantiate scanner.
5. Execute scan.

6. Render findings.
7. Exit.

Figure 13.2 illustrates this workflow.

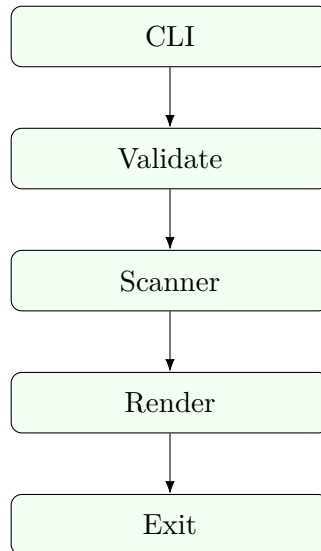


Figure 13.2: CLI execution sequence.

13.4 scanner.py

13.4.1 Purpose

The scanner constitutes the computational core of SafeCommit.

Its responsibility is to transform a filesystem hierarchy into a structured collection of findings.

13.4.2 Responsibilities

The scanner

- traverses directories,
- discovers files,
- loads text,
- evaluates detection rules,
- records findings,
- returns structured results.

13.4.3 Design Considerations

The scanner intentionally remains independent of

- Rich,
- Typer,
- terminal rendering,
- package metadata.

This separation allows future interfaces, such as graphical applications or web services, to reuse the same scanning engine without modification.

13.5 `patterns.py`

13.5.1 Purpose

The `patterns.py` module contains the complete catalogue of credential detection rules supported by SafeCommit.

Centralising detection rules within a single module simplifies

- maintenance,
- testing,
- extension,
- documentation.

13.5.2 Responsibilities

Responsibilities include

- defining regex patterns,
- assigning severity,
- documenting providers,
- organising credential families.

The scanning engine should never contain provider-specific regular expressions directly.

13.6 `utils.py`

13.6.1 Purpose

Utility functions encapsulate reusable helper logic shared by multiple modules.

Typical responsibilities include

- formatting,
- path manipulation,
- helper validation,
- reusable processing logic.

Separating utilities from business logic reduces duplication throughout the codebase.

13.7 Module Dependencies

Figure 13.3 presents the conceptual dependency graph.

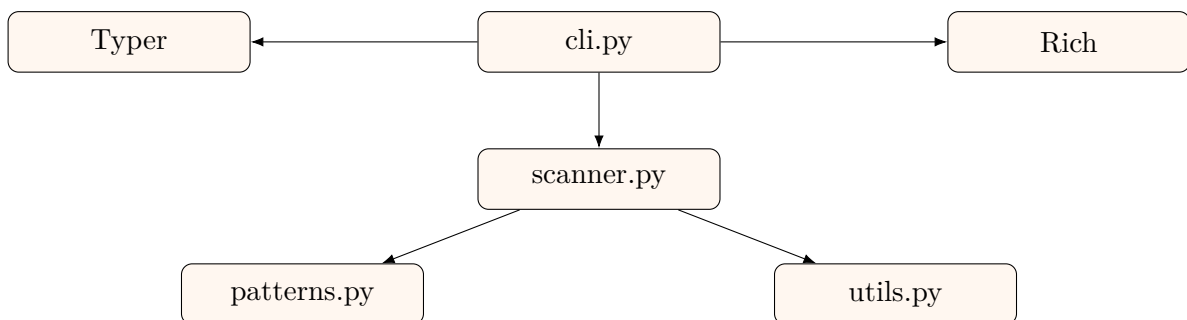


Figure 13.3: High-level dependency graph.

13.8 Design Patterns

Several software engineering principles influenced the implementation.

13.8.1 Separation of Concerns

Each module performs one primary responsibility.

13.8.2 Loose Coupling

Modules communicate through clearly defined interfaces rather than shared implementation details.

13.8.3 High Cohesion

Closely related functionality remains grouped within the same module.

13.8.4 Extensibility

Adding support for additional credential providers generally requires extending only the detection catalogue rather than modifying the scanning engine.

13.9 Code Quality Considerations

The following practices are recommended for future development.

- Static type hints.
- Descriptive docstrings.
- Comprehensive unit tests.
- Consistent formatting.
- Limited function complexity.
- Minimal global state.
- Explicit error handling.

Following these practices promotes maintainability and simplifies long-term project evolution.

14 Future Roadmap

14.1 Vision

The long-term objective of SafeCommit is to become a lightweight, cross-platform, developer-friendly secret scanning utility suitable for both individual developers and continuous integration environments.

Future development will prioritise accuracy, usability, and maintainability while preserving the project's lightweight philosophy.

14.2 Planned Features

Potential enhancements include

- Git pre-commit hook integration.
- Native GitHub Actions support.
- GitLab CI integration.
- Configurable ignore files.
- Custom detection rules.
- JSON report generation.
- SARIF export.
- HTML reporting improvements.
- YAML configuration.
- TOML configuration.
- Incremental repository scanning.
- Parallel file processing.
- Binary file detection.
- Additional cloud providers.
- Secret entropy analysis.

Implementation priority will depend upon community feedback and practical development resources.

14.3 Long-Term Goals

The project aspires to

- support a broader range of credential providers,
- improve detection precision,
- reduce false positives,
- maintain minimal dependencies,
- encourage community contributions,
- provide stable public releases.

These objectives guide the continued evolution of SafeCommit while preserving its original philosophy of simplicity and reliability.

Glossary

API Key A credential used to authenticate requests to a remote service.

Credential Any value used to verify identity or authorise access.

False Positive A benign value incorrectly identified as a secret.

False Negative A genuine secret that is not detected.

PAT Personal Access Token.

Regex Regular Expression used for pattern matching.

Secret Sensitive information that must remain confidential.

Severity A classification representing the potential impact of credential exposure.

References

Bibliography

- [1] Python Software Foundation. *Python Documentation*.
- [2] Will McGugan. *Rich Documentation*.
- [3] Typer Project. *Typer Documentation*.
- [4] OWASP Foundation. *OWASP Secrets Management Cheat Sheet*.
- [5] Amazon Web Services. *AWS Identity and Access Management Documentation*.
- [6] GitHub Documentation. *Personal Access Tokens*.
- [7] Stripe Documentation. *API Authentication*.

License

SafeCommit is distributed under the terms of the MIT License.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, subject to the conditions specified in the accompanying LICENSE file.

End of Document